

智能家居控制系统

——产品总体设计文档

产品名称：OmniHome 智能家居控制系统

文档类型：产品总体设计文档

文档版本：V1.0

编制日期：2026 年 6 月 18 日

目录

1	引言	7
1.1	编写目的	7
1.2	项目背景	7
1.3	设计目标	8
1.3.1	实现统一控制入口	8
1.3.2	实现完整控制闭环	8
1.3.3	实现状态同步与本地缓存	8
1.3.4	实现可扩展架构	8
1.4	目标用户	8
1.4.1	家庭主用户	8
1.4.2	家庭成员	9
1.5	设计原则	9
1.5.1	分层解耦原则	9
1.5.2	统一契约原则	9
1.5.3	增量同步原则	9
1.5.4	可替换性原则	9
2	系统总体设计	9
2.1	系统定位	9
2.2	建设思路	10

2.3	系统总体架构	10
2.3.1	客户端应用层	10
2.3.2	控制中心服务层	11
2.3.3	共享契约层	11
2.3.4	数据与设备层	11
2.4	总体设计原则详述	11
2.4.1	前端轻业务、后端重编排	11
2.4.2	结构化状态而非散乱变量	11
2.4.3	版本驱动同步而非全量拉取	12
2.4.4	事件推送补充轮询	12
2.5	系统边界	12
2.5.1	已纳入系统边界的内容	12
2.5.2	尚未纳入系统边界的内容	12
2.6	总体业务流程	13
3	系统架构设计	13
3.1	架构概述	13
3.2	前端架构设计	14
3.2.1	前端架构目标	14
3.2.2	页面层	14

3.2.3	控制层与视图模型层	14
3.2.4	仓储层	15
3.2.5	前端状态回写机制	15
3.3	后端架构设计	16
3.3.1	后端职责定位	16
3.3.2	路由模块化设计	16
3.3.3	设备注册表	17
3.3.4	场景注册表与自动化能力	17
3.3.5	命令历史	17
3.3.6	数据库服务	17
3.4	共享契约设计	18
3.4.1	共享契约的必要性	18
3.4.2	共享契约的主要内容	18
3.4.3	共享契约带来的收益	18
3.5	数据与设备层设计	19
3.5.1	SQLite 的定位	19
3.5.2	模拟设备层的定位	19
3.5.3	可替换性设计	19
4	功能模块设计	20

4.1	首页总览模块	20
4.2	设备控制模块	21
4.3	场景与自动化模块	21
4.4	家庭与通知模块	21
5	数据库设计	21
5.1	设计目标	21
5.2	数据库选型	21
5.2	主要数据表	22
5.3	数据设计特点	22
5.4	数据关系分析	23
6	接口设计	23
6.1	接口分层说明	23
6.2	接口设计特点	24
7	运行设计	24
7.1	读取链路	25
7.2	控制链路	25
7.3	同步链路	25
7.4	实时链路	26
8	安全设计	26

8.1	命令签名机制	26
8.2	重放保护机制	26
9	部署设计	27
9.1	客户端部署	27
9.2	服务端部署	27
9.3	扩展部署方向	28
10	总结与展望	28

1 引言

1.1 编写目的

本文档用于对 OmniHome 智能家居控制系统的总体设计方案进行系统化说明，重点描述系统的分层架构、功能模块、数据组织、接口约束、运行链路、安全机制与部署方式，为后续扩展提供统一的技术依据。

1.2 项目背景

随着家庭数字化与物联网设备普及，门锁、灯具、空调、摄像头、空气传感器、窗帘、插座、场景面板等智能设备正在逐步进入普通家庭。设备种类增多虽然提升了家庭自动化和便利性，但也带来了新的管理问题：

- 1) 设备入口分散。不同厂商、不同协议和不同终端往往对应不同 App 或不同控制入口，用户在实际使用过程中需要频繁切换应用，难以形成统一的家庭控制体验。
- 2) 设备状态割裂。灯光页面能看到灯的状态，门锁页面能看到门锁状态，但家庭整体层面缺少统一的状态总览，用户很难迅速判断当前家中是否存在异常，例如某个房间灯未关闭、某个设备掉线、某个摄像头未在录制。
- 3) 控制反馈滞后。很多简单演示型系统缺少完整处理命令执行结果、状态回写、失败反馈、历史留痕和实时同步，导致系统看起来能用，但并不真正可靠。
- 4) 扩展成本高。如果系统前期没有统一契约、没有明确的分层、没有考虑缓存与同步，一旦后续新增设备类型、自动化规则、权限机制或多终端访问能力，就容易陷入“牵一发而动全身”的重构困境。

基于上述问题，本项目设计并实现了 OmniHome 智能家居控制系统。系统以 **OpenHarmony 客户端** 作为统一交互入口，以 **Fastify 控制中心服务** 作为业务中枢，以共享契约层作为前后端协同边界，以 **SQLite** 和 **版本化同步机制** 作为本地状态与数据一致性的基础，形成一个适合后续扩展的智能家居系统原型。

1.3 设计目标

本系统的建设目标可概括为以下四个方面：

1.3.1 实现统一控制入口

系统需要为用户提供单一应用入口，使其可以在同一控制界面内完成家庭设备总览、门禁管理、摄像监控、环境控制、家庭广播、场景执行与自动化管理，而不必依赖多个分散入口。

1.3.2 实现完整控制闭环

系统不仅要支持发起控制命令，还要支持命令签名、命令验证、命令执行、设备状态更新、历史记录写入、实时事件广播以及前端状态回显，形成闭环式业务处理链路。

1.3.3 实现状态同步与本地缓存

系统需要支持前端本地缓存、增量同步和实时推送机制，避免单纯依赖全量刷新和频繁轮询。这样既可以提高用户体验，也可以为将来离线容错与弱网容错提供基础。

1.3.4 实现可扩展架构

系统应在初期就对设备模型、命令结构、场景定义、接口组织和数据库结构做抽象，使后续接入更多设备、更多页面和更多家庭规则时能够以较低成本扩展。

1.4 目标用户

系统的直接使用者主要包括以下几类：

1.4.1 家庭主用户

家庭主用户通常负责家庭设备的整体管理，需要查看全屋设备在线状态、调整核心设备参数、维护家庭场景和管理自动化规则。

1.4.2家庭成员

普通家庭成员更关注简洁、直观和高频操作，例如开关灯、调整空调温度、查看门锁状态、接收家庭广播等，因此系统需要提供清晰的模块导航与较低的操作门槛。

1.5 设计原则

为了使系统具备工程化扩展空间，本文档采用以下设计原则指导总体设计：

1.5.1分层解耦原则

前端展示、前端控制逻辑、接口访问逻辑、后端路由、领域服务、数据库访问和设备适配应尽量分层组织，减少彼此之间的直接耦合。

1.5.2统一契约原则

设备状态、设备命令、场景定义和安全信封等核心数据结构在系统中必须统一定义，避免前后端各自理解不同造成联调错误。

1.5.3增量同步原则

系统不依赖粗放式全量刷新，而是通过版本号和增量数据包完成状态同步，提高效率并降低前端更新复杂度。

1.5.4可替换性原则

模拟设备层必须被设计为可替换结构，使系统在未来接入真实网关、真实驱动或真实物联网平台时，不必重写上层业务逻辑。

2 系统总体设计

2.1 系统定位

OmniHome 智能家居控制系统定位为一个“**集中控制型智能家居中控原型系统**”。这里的“集中控制型”包含两层含义：

- 1) 控制入口集中。原本散落在多个页面、多个设备逻辑甚至多个应用中的控制行为，被统一抽象到一个中控系统中完成组织和呈现。
- 2) 状态理解集中。系统并不只是机械地透传设备状态，而是通过首页摘要、模块总览、历史记录、事件推送等方式，把多个设备状态组织成对用户有意义的家庭状态视图。

因此，本系统不是一个简单的“设备面板集合”，而是一个具备家庭状态中枢特征的系统。它强调：

- 1) 从页面到服务端的数据闭环。
- 2) 从设备状态到业务摘要的聚合能力。
- 3) 从单次控制到历史留痕的可追踪能力。
- 4) 从当前状态到实时变化的可同步能力。

2.2 建设思路

系统总体设计的建设思路不是“先做界面，再补后端”，而是采用“产品场景驱动 + 架构主线先行”的方式展开。也就是说：

- 1) 先明确系统支持哪些核心家庭场景。
- 2) 再围绕这些场景抽象出统一的设备模型和命令模型。
- 3) 在此基础上设计控制中心服务、同步机制和实时推送机制。
- 4) 最后由前端按功能域消费这些能力。

这种顺序的优势在于，页面不会成为系统唯一中心，后端也不会只是被动的“接口仓库”，而是能够从一开始就形成围绕设备、命令、状态和同步构建的完整系统骨架。

2.3 系统总体架构

从总体上看，系统由以下四个核心层次构成：

2.3.1 客户端应用层

客户端应用层运行在 OpenHarmony 环境中，主要负责界面呈现、用户交互、页面导航、局部状态维护和控制指令触发。该层面向用户，是系统的直接使用界面。

2.3.2控制中心服务层

控制中心服务层由 Fastify 提供支撑，是系统的业务核心。它负责管理设备注册表、场景注册表、房间注册表、命令历史、数据库版本同步、命令验签以及 WebSocket 事件广播。

2.3.3共享契约层

共享契约层用于统一定义前后端共同依赖的设备模型、命令结构和安全信封结构。该层是系统中“语义一致性”的关键保障。

2.3.4数据与设备层

数据与设备层包含 SQLite 数据库、设备状态快照、版本元数据以及模拟设备和适配器实现。它为业务层提供状态来源、持久化支撑和设备执行基础。

2.4 总体设计原则详述

2.4.1前端轻业务、后端重编排

系统并没有把复杂逻辑堆到前端，而是让前端聚焦于组织用户意图、发起请求和展示结果；真正的状态更新、命令执行、事务控制和事件广播由后端完成。这样设计的好处是：

- 1) 页面逻辑更容易维护。
- 2) 命令执行规则统一，不容易出现不同页面行为不一致的问题。
- 3) 未来新增客户端时，可以复用同一套控制中心能力。

2.4.2结构化状态而非散乱变量

系统状态并不是各页面自己维护一批零散变量，而是围绕“设备、房间、场景、自动化、历史记录”这些实体建立结构化模型。结构化状态能够天然支持同步、持久化和扩展。

2.4.3 版本驱动同步而非全量拉取

若每次进入页面都全量拉取所有家庭数据，一方面开销较大，另一方面无法优雅处理多页面刷新和后台状态变化。当前系统通过 `global\_version` 和各表 `version` 字段实现版本驱动同步，使客户端只获取变化部分，从而提高效率。

2.4.4 事件推送补充轮询

系统并未完全依赖前端重复发起请求来获取状态变化，而是通过 WebSocket 将关键状态变化主动广播给客户端。这一设计尤其适用于门锁状态变化、场景执行结果回写、设备上下线等需要快速反馈的场景。

2.5 系统边界

本系统边界如下：

2.5.1 已纳入系统边界的内容

- 1) OpenHarmony 客户端页面与交互流程。
- 2) Fastify 控制中心服务。
- 3) 设备、场景、自动化、房间、历史等核心业务结构。
- 4) HMAC 命令签名与重放保护。
- 5) SQLite 持久化、版本同步与 WebSocket 推送。
- 6) 模拟设备层与演示异常注入接口。

2.5.2 尚未纳入系统边界的内容

- 1) 商业级用户身份认证与权限中心。
- 2) 真实第三方物联网平台对接。

- 3) 云端多家庭长期数据托管。
- 4) 复杂规则引擎、AI 场景推荐和跨家庭共享能力。

明确边界有助于保证文档真实性，也能让后续扩展规划更加清楚。

2.6 总体业务流程

从用户视角看，系统总体业务流程如下：

- 1) 用户打开客户端并进入首页。
- 2) 首页读取摘要数据、设备列表和模块状态。
- 3) 用户进入门禁、气候、摄像头、自动化等具体模块。
- 4) 当用户发起设备控制或场景执行时，前端先组织命令，再请求签名，再正式提交执行。
- 5) 后端在执行后更新数据库状态，并通过实时通道通知前端。
- 6) 前端将事件落地为新的界面状态，同时保留历史记录供用户回看。

这一流程体现出系统是“以状态为中心”的，而非“以页面按钮为中心”的。

3 系统架构设计

3.1 架构概述

系统总体采用“客户端应用层 + 控制中心服务层 + 共享契约层 + 数据与设备层”的四层架构。该架构的核心思想是：通过明确边界，让每一层专注于自己的职责，并通过统一契约和标准接口完成协作。

从逻辑关系上看，系统的主链路可抽象为：

`ArkTS View -> Controller/ViewModel -> Repository -> DeviceApi -> Fastify Routes -> Registry / Database / Simulator -> WebSocket / Sync -> 前端状态回写`

这条链路完整描述了从页面操作到后端业务执行，再到前端状态刷新的一整套机制。

3.2 前端架构设计

3.2.1 前端架构目标

前端架构的主要目标不是“把页面做出来”，而是满足以下要求：

- 1) 让页面层保持较高可读性。
- 2) 让业务逻辑尽量从视图中抽离。
- 3) 让数据访问逻辑具备统一入口。
- 4) 让本地缓存、同步数据和实时事件能汇聚到一致的数据处理流程中。

3.2.2 页面层

页面层直接对应用户可见界面，是用户操作与视觉反馈的承载面。项目中主要包括：

- 1) 首页总览页面。
- 2) 门禁控制页面。
- 3) 摄像头监控页面。
- 4) 气候控制页面。
- 5) 家庭成员与广播页面。
- 6) 自动化与场景页面。
- 7) 通知与历史页面。

页面层的职责主要包括展示数据、响应用户点击、触发导航和呈现错误反馈。页面层不应直接拼装复杂请求，也不应直接处理数据库或同步策略。

3.2.3 控制层与视图模型层

控制层和视图模型层的意义在于把“页面要做什么”与“系统应该怎么做”区分开来。比如：

- 1) 用户点击开灯，本质上不是“改一个 UI 布尔值”，而是“构造一次设备开关命令，并走安全链路提交到服务端”。
- 2) 用户进入首页，本质上不是“读取一个接口”，而是“加载摘要、设备列表和状态统计，然后组织成首页所需的视图模型”。

因此，控制层与视图模型层承担页面与仓储之间的桥梁作用，负责：

- 1) 解释用户意图。
- 2) 组织页面依赖的数据结构。
- 3) 调用仓储层统一访问远端与本地数据。
- 4) 对错误、空态、加载态进行统一处理。

3.2.4 仓储层

仓储层是前端架构中非常关键的一层。它的价值在于：

- 1) 将本地缓存读取与远程接口访问统一封装。
- 2) 为不同页面提供稳定的数据访问入口。
- 3) 接收同步结果和实时事件，完成状态整合。

如果没有仓储层，页面可能会直接调用多个 API，导致数据来源散乱、更新时机不一致、重用困难。仓储层的存在可以让“页面如何读数据”和“系统数据从哪里来”解耦。

3.2.5 前端状态回写机制

前端并不是在命令发出后立即假设操作成功，而是在后端确认执行并更新状态后，通过同步或事件回写刷新 UI。这种模式比“前端乐观直接改界面”更适合智能家居场景，因为物理世界控制存在失败、拒绝和设备离线等风险。

3.3 后端架构设计

3.3.1 后端职责定位

控制中心服务不是“简单 API 转发层”，而是系统的控制中枢。其职责包括：

- 1) 提供 REST API 和 WebSocket 入口。
- 2) 持有设备注册表和业务注册表。
- 3) 对命令执行进行校验与编排。
- 4) 维护数据库状态与版本信息。
- 5) 维护场景和自动化业务逻辑。
- 6) 维护命令历史与异常演示能力。

3.3.2 路由模块化设计

系统在 `buildApp()` 中注册多个路由模块，包括：

- `devices` 路由：设备列表、设备详情、首页摘要、设备房间归属。
- `access` 路由：门禁状态、数字钥匙和访客授权。
- `camera` 路由：摄像头概览和录制切换。
- `climate` 路由：气候概览、目标温度和运行模式。
- `family` 路由：家庭成员动态和广播消息。
- `scenes` 路由：场景查询、维护、启停与执行。
- `automations` 路由：自动化规则查询和维护。
- `commands` 路由：签名、控制命令和历史记录。
- `sync` 路由：按版本返回增量数据。
- `websocket` 路由：实时事件通道。

- `demo` 路由：环境修改、异常注入和安全拒绝演示。

这种设计的优点是，每条业务线都有明确边界，开发、测试和扩展时都更加清晰。

3.3.3设备注册表

设备注册表负责维护当前系统已知设备及其状态，是设备层和接口层之间的重要桥梁。它的存在使系统不必把设备状态完全散落在接口处理函数中，而是以注册表形式集中组织。

注册表机制的主要价值包括：

- 1) 提供快速设备查找能力。
- 2) 提供统一设备状态更新入口。
- 3) 为摘要统计、设备列表和控制执行提供一致的状态源。

3.3.4场景注册表与自动化能力

场景注册表维护可执行场景及其启停状态。场景与自动化虽然都属于“多设备联动能力”，但语义不同：

- 1) 场景偏“显式触发”，例如用户点击“离家模式”。
- 2) 自动化偏“条件触发”，例如晚上十点自动锁门。

在架构上将两者分开，有利于后续扩展更复杂的规则引擎。

3.3.5命令历史

命令历史用于记录用户发起的设备控制和系统返回的执行结果，是通知页和系统可追踪性的基础。历史记录不仅对用户可见，也对系统调试具有重要意义。

3.3.6数据库服务

数据库服务不是简单的 CRUD 包装，而是承担如下作用：

- 1) 管理版本号递增。
- 2) 组织增量同步数据。
- 3) 确保事务更新原子性。
- 4) 为前端同步链路提供结构化响应。

3.4 共享契约设计

3.4.1 共享契约的必要性

在智能家居系统中，如果前端自己定义一套设备模型，后端自己维护另一套设备模型，很容易导致字段含义不一致、命令名称不统一、时间戳格式不一致等问题。共享契约层就是为解决这种问题而存在的。

3.4.2 共享契约的主要内容

共享契约层主要包括以下类型：

- 设备描述类型。
- 设备状态类型。
- 增强设备快照类型。
- 设备命令类型。
- 场景描述类型。
- 场景执行结果类型。
- 命令历史条目类型。
- 安全信封类型。

3.4.3 共享契约带来的收益

共享契约层为系统带来的收益主要体现在：

- 1) 前后端开发语言虽然不同，但能围绕统一数据结构协作。

- 2) 新增设备或新命令时，只需先扩展契约层，再扩展两端实现，过程更规范。
- 3) 测试时可以针对契约做结构性校验，减少隐式错误。

3.5 数据与设备层设计

3.5.1 SQLite 的定位

SQLite 在当前系统中的定位不是“大型生产数据库替代品”，而是轻量级、可嵌入、适合本地版本化管理的数据载体。

3.5.2 模拟设备层的定位

模拟设备层是系统能形成完整闭环的重要前提。没有模拟设备，系统只能“看起来能发请求”，却不能真实反映控制结果。模拟设备层让后端在收到命令后可以更新门锁、灯光和空调状态，从而保证控制链路完整。

3.5.3 可替换性设计

模拟设备层通过设备模拟器或适配器方式接入，这意味着未来若要接入真实设备，只需替换底层执行逻辑，而不必重写上层接口、同步、历史记录与事件推送机制。

4 功能模块设计

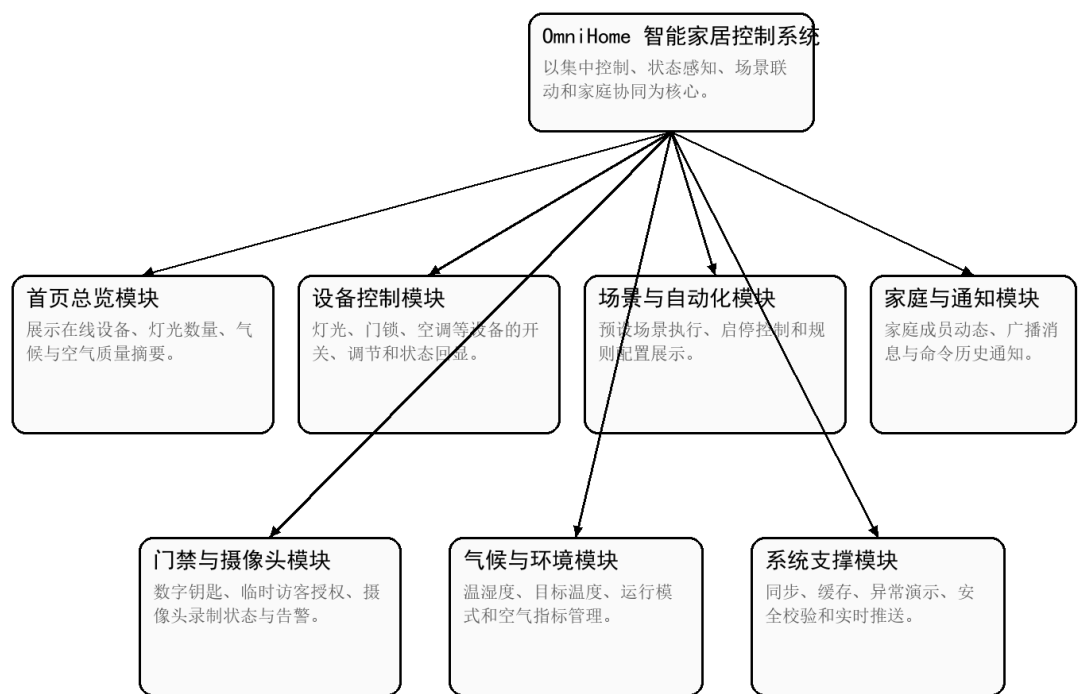


图 4-1 功能模块结构图

4.1 首页总览模块

首页总览模块是系统的入口页，也是最能体现“集中控制平台”价值的页面。其设计重点不是展示所有细节，而是用有限信息帮助用户快速判断家庭整体状态。

首页总览模块主要包括：

- 在线设备数量。
- 灯光数量和运行概况。
- 温湿度与空气质量。
- 重点告警或异常状态。
- 常用业务模块的快捷入口。

该模块的设计原则是“信息摘要优先”。用户进入首页后，应先判断家庭整体是否正常，再决定是否进入具体模块。

4.2 设备控制模块

设备控制模块面向门锁、灯光、空调等设备提供统一的控制入口，支持开关、温度设定、亮度调节、色温切换和门锁控制等能力。所有控制行为均通过命令接口提交，并在服务端执行后反馈到本地状态。

4.3 场景与自动化模块

场景与自动化模块支持预定义场景的查看、启停和执行，以及自动化规则的创建、启用和维护。该模块的价值在于将多个设备命令组合成可复用的家庭控制策略，提高系统的业务表达能力。

4.4 家庭与通知模块

家庭与通知模块承担家庭成员活动展示、全屋广播以及命令历史回显等功能，使系统不再局限于单一设备管理，而是具备家庭协同与状态留痕能力。

5 数据库设计

5.1 设计目标

数据库设计目标主要包括：

- 保存核心业务实体状态。
- 支持版本化增量同步。
- 支持命令历史追踪。
- 支持轻量部署和本地演示。
- 为后续替换为更大规模数据库打下结构基础。

5.2 数据库选型

当前系统使用 **SQLite** 作为数据库，原因包括：

- 部署简单，无需单独安装服务。
- 对课程原型和本地演示足够稳定。
- 与 Node.js 生态适配较好。
- 适合保存结构化状态与少量历史数据。

5.2 主要数据表

表名	主要字段	设计作用
metadata	key、value	保存 global_version 、 schema_version 等元数据，用于版本控制与迁移管理。
devices	id 、 name 、 type 、 room_id 、 state_json、 updated_at、 version、 is_deleted	保存设备状态快照，是首页总览、设备详情和同步接口的重要数据来源。
rooms	id 、 name 、 icon 、 built_in 、 updated_at、 version、 is_deleted	保存房间信息，为设备归属和页面组织提供支撑。
scenes	id 、 name 、 description 、 enabled 、 trigger_json 、 repeat_json、 commands_json	保存场景定义、启用状态及动作集合。
automations	id 、 icon 、 name 、 trigger_type 、 trigger_json 、 action_json、 enabled、 version	保存自动化规则及触发动作配置。
history	id 、 request_id 、 device_id 、 command_name 、 status 、 message 、 created_at	保存命令执行结果和错误反馈，支撑通知与审计。

5.3 数据设计特点

数据库设计中最关键的点在于引入 global_version 和各业务表 version 字段，用于支撑 /api/sync 增量同步接口。每当设备、房间、场景或自动化对象发生变化时，系统

通过事务同步提升版本号，从而使前端能够按版本拉取变化数据，避免全量同步带来的性能开销。

5.4 数据关系分析

系统当前并未采用复杂外键约束体系，但从逻辑关系上看，实体间存在如下关系：

- 1) 一个房间可以关联多个设备。
- 2) 一个场景可以关联多个设备动作。
- 3) 一个自动化规则可以绑定多个触发条件和动作定义。
- 4) 一条历史记录通常对应一次命令执行。

当前系统以轻量实现为主，因此部分复杂关系通过 JSON 字段表达。这种设计兼顾了灵活性和实现成本。

6 接口设计

6.1 接口分层说明

系统接口设计围绕业务域进行分组，包括设备与摘要接口、命令控制接口、场景与自动化接口、垂直领域接口、同步接口和实时推送接口。接口分组有助于提高系统可维护性，也便于前端页面按模块装配数据。

接口类别	代表接口	作用说明
设备与摘要接口	GET /api/devices; GET /api/devices/:deviceId; GET /api/summary	为首页总览和设备详情提供基础状态数据。
命令控制接口	POST /api/demo/sign-command; POST /api/commands; GET /api/commands/history	完成命令签名、命令执行与历史回显。
场景与自动化接口	GET/POST/PUT/DELETE /api/scenes; GET/POST/PUT/DELETE	支撑场景和自动化规则的查询、维护与执行。

	/api/automations	
垂直领域接口	GET /api/access; GET /api/cameras; GET /api/family; GET /api/climate	为门禁、摄像头、家庭和气候模块提供专属数据。
同步接口	GET /api/sync?lastVersion=n	返回版本号之后的增量数据，用于前端缓存刷新。
实时接口	GET /ws/events	建立 WebSocket 通道，将设备状态变化广播给客户端。

6.2 接口设计特点

命令类接口采用“先签名、后执行”的双阶段设计，避免客户端直接拼装不受保护的命令。同步类接口采用版本号增量拉取策略，与 WebSocket 实时事件共同组成“主动推送 + 按需补偿”的状态一致性机制。

7 运行设计

系统运行设计重点描述从用户操作触发到界面状态回写的全过程。与单纯页面型系统不同，本系统的数据流既包含读取路径，也包含控制路径、同步路径和实时路径。

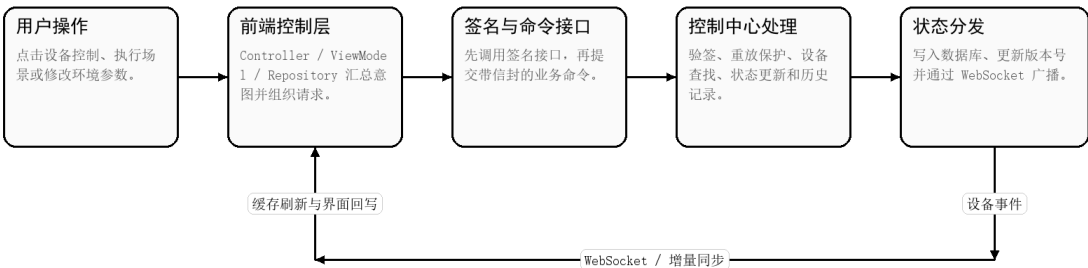


图 7-1 核心控制与同步数据流图

7.1 读取链路

读取链路用于页面初始加载和主动刷新。其过程如下：

- 1) 用户进入某个页面。
- 2) 页面层调用控制层或视图模型层。
- 3) 控制层调用仓储层。
- 4) 仓储层决定从本地缓存还是远端接口读取数据。
- 5) 后端返回结果后，前端将其映射为页面所需状态。
- 6) 页面完成渲染。

7.2 控制链路

控制链路是系统最关键的运行链路之一，其过程如下：

- 1) 用户在页面中点击某个设备控制按钮。
- 2) 页面把动作交给控制层处理。
- 3) 控制层构造业务命令。
- 4) 前端调用签名接口获取安全信封。
- 5) 前端提交正式控制请求到 `/api/commands`。
- 6) 服务端验签并执行重放保护校验。
- 7) 服务端查找设备并调用模拟器执行。
- 8) 服务端更新数据库状态与版本号。
- 9) 服务端写入命令历史。
- 10) 服务端广播状态变化事件。
- 11) 前端收到事件后刷新界面。

7.3 同步链路

同步链路用于保证前端本地状态与服务端状态最终一致。其过程如下：

- 1) 前端保存 `lastVersion`。

- 2) 当前端需要补偿同步时，调用 ``/api/sync``。
- 3) 服务端根据版本号筛选变化记录。
- 4) 返回变化实体以及当前版本号。
- 5) 前端将变化数据落地到本地缓存。
- 6) 前端更新自身的同步水位。

7.4 实时链路

实时链路用于提高反馈速度，其过程如下：

- 1) 前端在应用运行期间建立 WebSocket 连接。
- 2) 服务端保存该连接。
- 3) 当设备状态变化时，服务端广播事件。
- 4) 前端接收到事件后更新本地状态与界面。

8 安全设计

8.1 命令签名机制

系统在命令执行前通过 `/api/demo/sign-command` 对设备命令进行 HMAC-SHA256 签名，生成包含命令、时间戳、随机 `nonce` 和摘要信息的安全信封。客户端随后将完整信封提交到 `/api/commands`，由服务端进行统一验证。

8.2 重放保护机制

服务端通过 `ReplayGuard` 记录 `nonce` 和时间窗口，对重复提交或超时命令进行拒绝，从而降低控制链路受到重放攻击的风险。该机制对于智能家居控制场景尤其重要，因为门锁、灯光和空调控制均属于可执行的物理世界指令。

9 部署设计

系统部署主要面向开发与演示环境展开，采用“OpenHarmony 客户端 + 本地控制中心服务 + SQLite 数据库 + 模拟设备层”的轻量级部署结构。该结构部署成本低、链路清晰。

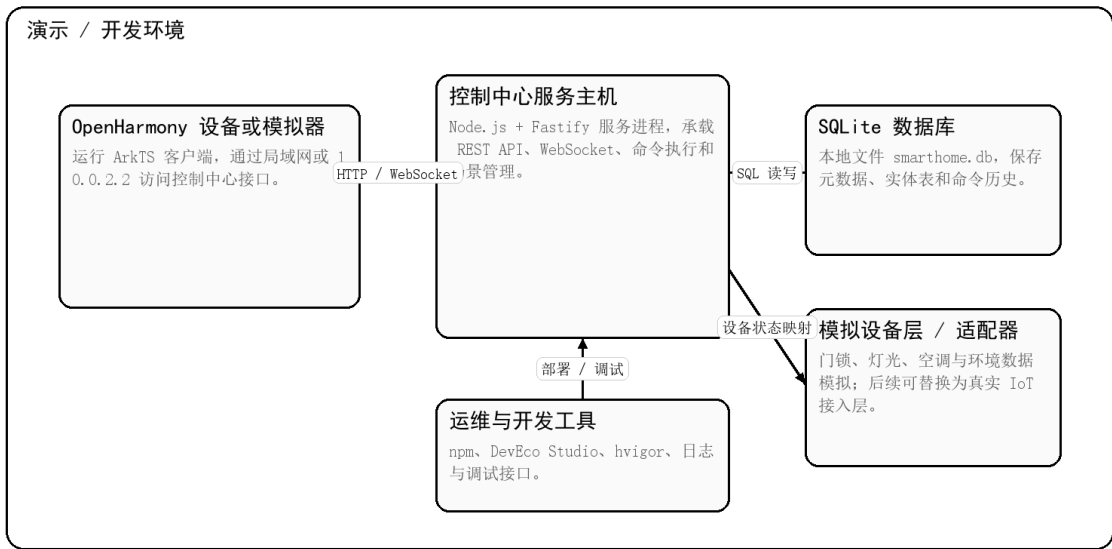


图 9-1 系统部署图

9.1 客户端部署

客户端运行于 OpenHarmony 模拟器或真实设备，通过 DevEco Studio 构建 HAP 包后安装。演示环境下，客户端默认通过 10.0.2.2 访问宿主机控制中心服务；若部署到真机环境，则需要将接口地址调整为局域网主机地址。

9.2 服务端部署

控制中心服务运行于 Node.js 环境，依赖 Fastify、better-sqlite3 和共享契约包。服务端启动后同时提供 REST API 与 WebSocket 通道，数据库文件 smarthome.db 与服务进程共机部署。

9.3 扩展部署方向

后续如果接入真实物联网设备，可在保持上层控制接口不变的前提下，将模拟设备层替换为真实设备网关或驱动适配层；同时可将 SQLite 迁移至独立数据库服务，以增强并发处理能力和部署弹性。

10 总结与展望

本系统在总体设计上形成了较完整的智能家居控制原型架构，既具备集中控制、场景联动、状态同步、历史留痕和安全控制等核心能力，也通过共享契约、版本同步和实时推送机制体现了较清晰的工程化思路。

未来可进一步扩展真实设备接入、用户身份认证、多家庭空间隔离、规则引擎增强和云边协同能力，使系统从课程原型逐步演进为更加贴近真实应用场景的智能家居控制平台。